Sim-to-Real Evaluation of a Pretrained 3D Object Detector in a ROS 2 / NVIDIA Isaac Sim Pipeline

Carlos Gonzales | CMPE 249 | 7/12/2026

Project Option Selection

For this project I am choosing Option 1 (Apply & Evaluate). Specifically I plan to take an existing pretrained 3D object detector, wrap it in a ROS 2 node, run it on both simulated data from NVIDIA Isaac Sim and real data from nuScenes, and evaluate its accuracy, latency, and resource usage.

Motivation and Use Case

Simulation has become a normal part of how autonomous driving software gets developed and tested. It is safer and cheaper than road testing, and it lets you set up rare scenarios on demand. This workflow assumes a perception model behaves roughly the same on simulated input as it does on real camera data, and that assumption often fails. Rendered images differ from real ones in lighting, texture, and sensor characteristics, so a detector trained on real driving footage can lose accuracy in a simulator. This is commonly called the sim-to-real gap.

My question is how large that gap actually is for a modern pretrained detector, and where does it come from? Which object classes suffer? Does it worsen with distance or lighting? And if so, by how much? A simulator is a good place to study this because it knows the exact 3D pose of every object in the scene. We get evaluation labels for free and can rerun the same scene while changing one variable at a time. That kind of controlled experiment is not really possible with a fixed real-world dataset. The practical relevance is that anyone validating perception software in simulation needs some idea of how much to trust the numbers it produces.

Since the model will be running as an actual ROS 2 node rather than an offline script, I will also measure it as a system: end-to-end latency from sensor message to published detection, sustained frame rate, and CPU/GPU/memory load under different runtime configurations.

Selected Model(s)

The primary model is FCOS3D, a monocular camera-only 3D object detector. It takes a single RGB image and predicts 3D bounding boxes with class labels for cars, pedestrians, cyclists, and other road objects, including each box’s position, size, and orientation. I picked it because it is anchor free and fully convolutional, so the input/output interface stays simple, and because it's light enough to run at near realtime rates inside a ROS 2 node. Pretrained nuScenes weights are published through the MMDetection3D framework.

- Paper: Wang et al., *FCOS3D: Fully Convolutional One-Stage Monocular 3D Object Detection*, ICCV Workshops 2021. arxiv.org/abs/2104.10956
- Code and weights: MMDetection3D (OpenMMLab), github.com/open-mmlab/mmdetection3d

If the FCOS3D pipeline comes together ahead of schedule, I would like to also run BEVFormer, a transformer-based detector that fuses six camera views into a bird’s eye view representation. Wrapping it behind the same node interface would let me compare a monocular detector against a multiview one under identical conditions. With only four weeks remaining I am treating this strictly as a stretch goal, and it is the

first thing I will drop if anything slips. (Li et al., ECCV 2022, arxiv.org/abs/2203.17270; github.com/fundamentalvision/BEVFormer.)

For the model presentation I will cover the network structure (backbone, neck, detection head), what the model expects as input, and the output format in camera/ego coordinates.

Dataset and Input/Output

On the simulated side, I will build driving scenes in NVIDIA Isaac Sim with simulated RGB cameras publishing through the Isaac Sim ROS 2 bridge. Isaac Sim exposes exact ground truth poses for every object in the scene, which I will export and use as evaluation labels. On the real side, I will use nuScenes (v1.0), replayed as ROS 2 bags so the exact same detection node processes both sources. nuScenes ships with 3D box annotations and a devkit for computing the standard mAP and NDS metrics.

Planned ROS 2 interface for the detection node:

Direction	Topic	Message type	Content
Subscribe	/camera/front/image_raw	sensor_msgs/Image	RGB camera frames (sim or replayed nuScenes)
Subscribe	/camera/front/camera_info	sensor_msgs/CameraInfo	Camera intrinsics
Publish	/perception/detections_3d	vision_msgs/Detection3DArray	3D boxes with class and confidence
Publish	/perception/markers	visualization_msgs/MarkerArray	Boxes for live RViz2 visualization

The evaluation has three parts. First, detection accuracy on nuScenes validation scenes compared against matched Isaac Sim scenes, broken down by object class, distance, and lighting. Second, controlled sweeps in simulation where I vary one thing at a time, mainly lighting (day vs. dusk). Third, system measurements: per-frame node latency, throughput, and CPU/GPU/memory use across FP32 and FP16 configurations, with TensorRT if time allows. My gaming setup has an RTX 4090, which makes running Isaac Sim locally feasible, and I will use a rented cloud GPU instance (RunPod) as the second computing environment for the cross platform comparison.

Timeline (Milestones)

Week	Milestone
1 (Jul 13–19)	Set up ROS 2 (Humble) and MMDetection3D, get the pretrained FCOS3D weights running offline on sample images. Attempt the Isaac Sim ROS 2 bridge in parallel. Go/no go decision on Isaac Sim by the end of the week.
2 (Jul 20–26)	Wrap FCOS3D as a ROS 2 node that subscribes to the camera topics and publishes detections plus RViz2 markers, with live detections in RViz2. Replay nuScenes scenes as ROS 2 bags and confirm the same node runs on real data. Prepare the model presentation.
3 (Jul 27–Aug 2)	Build the evaluation tooling: run the nuScenes devkit evaluation for the real-data baseline, export Isaac Sim ground truth and implement box matching, and run the lighting sweep (day vs. dusk).

4 (Aug 3–9)	Latency, throughput, and resource profiling: FP32 vs. FP16 on the RTX 4090 and on the RunPod instance. TensorRT only if ahead of schedule.
Aug 10–13	Final analysis, report writing, and presentation prep with a live RViz2 demo.

One risk worth flagging: if the Isaac Sim bridge turns out to be unworkable in the first week, the fallback is to run the same node and evaluation entirely on nuScenes ROS 2 bags. That version loses the sim-to-real comparison but still satisfies all of the Option 1 requirements.